

# git.kit

Replicação de commits entre branches

Manual do Usuário

Aplicativo Windows (WPF / .NET)

Versão do manual: 1.0

# Manual do Usuário

Este manual descreve como usar o **git.kit** para replicar um commit de um branch em outro, resolver conflitos com o TortoiseGitMerge e publicar o resultado.

## Sumário

---

1. O que é o git.kit
2. Pré-requisitos
3. Visão geral da tela principal
4. Passo a passo da replicação
5. Estratégias de replicação
6. Branch de destino e criação automática
7. Resolução de conflitos
8. Envio via push
9. Situações especiais e mensagens
10. Perguntas frequentes (FAQ)
11. Glossário

## 1. O que é o git.kit

---

O **git.kit** é um aplicativo desktop (Windows, WPF) que automatiza a tarefa de **levar um commit específico de um branch para outro** dentro de um repositório git. Ele sempre trabalha sobre uma **cópia temporária** do repositório, de modo que o seu projeto de trabalho **não é alterado** (não troca de branch nem modifica arquivos).

Principais recursos:

- Clonar a partir de uma **URL** ou abrir a partir de um **caminho local**.
- Escolher o **branch de origem** e o **branch de destino** (existente ou novo).
- Replicar via **cherry-pick** ou **integração de diff**.
- Resolver conflitos em um **formulário dedicado** usando o **TortoiseGitMerge**.
- Concluir a replicação e **enviar (push)** o branch para o remote.

## 2. Pré-requisitos

| Item                                        | Necessário para                                                   |
|---------------------------------------------|-------------------------------------------------------------------|
| <b>git</b> instalado e no <code>PATH</code> | Todas as operações (o app usa a linha de comando do git).         |
| <b>TortoiseGit</b> (com TortoiseGitMerge)   | Resolução manual de conflitos. Opcional se nunca houver conflito. |
| Acesso ao <b>remote</b> (credenciais/SSH)   | Apenas para o passo de push.                                      |

**Dica:** se o TortoiseGit não for encontrado automaticamente, o app pedirá para você localizar o executável ( `TortoiseGitProc.exe` ou `TortoiseGitMerge.exe` ) na primeira vez que precisar dele.

### 3. Visão geral da tela principal

1. Repositório

URL ou caminho:

Iniciar

Local: C:\Users\Paulo Rogério\AppData\Local\Temp\git.kit\when.it-20260628-180006-f57813d07b514f55af54b85cf7727f07

Remote: git@github.com:exemplo/when.it.git

2. Branches e commit

Branch de origem:

Branch de destino (existente ou novo):

Atualizar

Estratégia:

3. Commit a replicar (do branch de origem)

| Hash    | Data             | Autor          | Mensagem                         |
|---------|------------------|----------------|----------------------------------|
| 1967a47 | 2026-06-28 16:00 | Paulo Oliveira | commit dev                       |
| a50c3e0 | 2026-06-27 18:00 | Paulo Oliveira | ajuste no parser de configuração |
| ba05fdc | 2026-06-26 18:00 | Maria Souza    | adiciona testes de integração    |
| 54b1c63 | 2026-06-25 18:00 | João Lima      | corrige validação de branch      |
| c0ffee0 | 2026-06-23 18:00 | Maria Souza    | refatora serviço de replicação   |

Replicar commit

Log de comandos git

```
$ clone git@github.com:exemplo/when.it.git C:\Users\Paulo Rogério\AppData\Local\Temp\git.kit\when.it-20260628-180006-f57813d07b514f55af54b8
Cloning into ...
done.
```

5 commit(s) do branch 'develop'.

Tela principal: repositório, branches/estratégia, lista de commits e log de comandos.

A janela principal é dividida em blocos numerados:

| Bloco                | Função                                                                                                 |
|----------------------|--------------------------------------------------------------------------------------------------------|
| 1. Repositório       | Campo "URL ou caminho" e botão <b>Iniciar</b> . Mostra o local da cópia (Local:) e o remote (Remote:). |
| 2. Branches e commit | Branch de origem, branch de destino (editável) e a estratégia de replicação.                           |
| 3. Commit a replicar | Lista de commits do branch de origem e os botões de ação.                                              |
| Log de comandos git  | Mostra cada comando git executado e sua saída.                                                         |
| Barra de status      | Mensagem do passo atual e indicador de progresso.                                                      |

## 4. Passo a passo da replicação

### 1 Informar o repositório

No campo **URL ou caminho**, informe uma das opções e clique em **Iniciar**:

- **URL** (https/ssh): o repositório é clonado em uma **pasta temporária única**.
- **Caminho local** de um repositório git: ele é validado e **clonado para uma pasta temporária** — assim o seu repositório original não troca de branch nem é alterado.

Em ambos os casos, os **branches são carregados automaticamente**.

### 2 Escolher origem e destino

- **Branch de origem**: selecione o branch que contém o commit a replicar. Os commits são carregados automaticamente na lista.
- **Branch de destino**: selecione um branch existente **ou digite um nome novo** (campo editável). Veja a seção 6 sobre criação automática.

### 3 Escolher a estratégia

Selecione **Cherry-pick** ou **Integração de diff** (ver seção 5).

### 4 Selecionar o commit e replicar

Na lista, clique no commit desejado e então em **Replicar commit**. O resultado pode ser:

| Resultado       | O que significa                                                                |
|-----------------|--------------------------------------------------------------------------------|
| Sucesso         | O commit foi aplicado e commitado automaticamente. Já é possível fazer o push. |
| Nada a replicar | As mudanças já existem no destino. Nada a fazer.                               |
| Conflitos       | Abre o formulário de resolução de conflitos (ver seção 7).                     |
| Falha           | Erro (ex.: árvore com alterações pendentes). A mensagem detalha o motivo.      |

### 5 Enviar (push)

Após o sucesso, clique em **Enviar branch (push)** (ver seção 8).

## 5. Estratégias de replicação

| Estratégia         | Como funciona                                                                              | Quando usar                                                                  |
|--------------------|--------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| Cherry-pick        | Replica o commit com <code>git cherry-pick -x</code> , preservando autoria e mensagem.     | Caso padrão — quando você quer "o mesmo commit" no outro branch.             |
| Integração de diff | Gera o diff do commit e aplica com <code>git apply --3way</code> , criando um novo commit. | Quando quer apenas as diferenças aplicadas, sem vincular ao commit original. |

## 6. Branch de destino e criação automática

O campo de destino é **editável**. Se você digitar um nome que **não existe**, o git.kit cria o branch automaticamente, escolhendo a base pelo **sufixo do nome**:

| Nome digitado                                                  | Base usada                                           |
|----------------------------------------------------------------|------------------------------------------------------|
| Termina em <code>dev</code> (ex.: <code>feature/x-dev</code> ) | <b>develop</b>                                       |
| Qualquer outro (ex.: <code>feature/x</code> )                  | <b>master</b> (com fallback para <code>main</code> ) |

**Observação:** a base é procurada tanto entre os branches locais quanto remotos ( `origin/...` ), o que é importante logo após o clone.

## 7. Resolução de conflitos

Quando o git não consegue aplicar o commit automaticamente, abre-se o **formulário de conflitos**, com um grid contendo:

Os arquivos abaixo estão em conflito. Clique em 'Resolver' para abrir o TortoiseGitMerge, salve a resolução e marque como resolvido; depois use 'Atualizar status'.

| Arquivo        | Tipo de conflito     | Código | Status    | Ação     |
|----------------|----------------------|--------|-----------|----------|
| src/Program.cs | Ambos modificaram    | UU     | Resolvido | Resolver |
| .gitignore     | Ambos modificaram    | UU     | Pendente  | Resolver |
| README.md      | Adicionado por ambos | AA     | Pendente  | Resolver |

Atualizar statusConcluir replicação

1 de 3 resolvido(s).

Formulário de conflitos: cada arquivo com seu tipo, status (Resolvido/Pendente) e o botão Resolver.

| Coluna           | Significado                                                            |
|------------------|------------------------------------------------------------------------|
| Arquivo          | Caminho do arquivo em conflito.                                        |
| Tipo de conflito | Ex.: "Ambos modificaram", "Adicionado por ambos", "Excluído por eles". |
| Código           | Código do git ( UU , AA , DU ...).                                     |
| Status           | <b>Pendente</b> ou <b>Resolvido</b> .                                  |
| Ação             | Botão <b>Resolver</b> (abre o TortoiseGitMerge).                       |

### Fluxo de resolução

1. Clique em **Resolver** na linha do arquivo. O **TortoiseGitMerge** abre mostrando três painéis: **(base)**, **(destino)** e **(origem)**.
2. Combine as alterações desejadas, **salve** e **marque como resolvido** no TortoiseGit.
3. Volte ao formulário: o status é reavaliado automaticamente (ou clique em **Atualizar status**). Arquivos resolvidos passam a **Resolvido**.
4. Quando todos estiverem resolvidos, clique em **Concluir replicação**. O app finaliza o commit ( `cherry-pick --continue` ou commit) e fecha o formulário.

**Atenção:** para trazer a mudança que está sendo replicada, aceite/combine o lado (**origem**) no TortoiseGitMerge. Se você mantiver apenas o lado (**destino**), o resultado fica idêntico ao destino e não haverá o que commitar (veja "commit vazio" na seção 9).

**Dica:** marcar como resolvido no TortoiseGit apenas faz o *stage* (git add) — não cria o commit. É o botão **Concluir replicação** que finaliza.

## 8. Envio via push

Após concluir a replicação, clique em **Enviar branch (push)**. Uma confirmação mostra o **remote de destino** antes do envio. O comando executado é `git push -u origin <branch>`.

| Origem do repositório           | Destino do push                                                                        |
|---------------------------------|----------------------------------------------------------------------------------------|
| Clone por <b>URL</b>            | O próprio remote real (origin).                                                        |
| Aberto por <b>caminho local</b> | O <b>remote real do repositório de origem</b> (o app reaponta o origin para essa URL). |

**Atenção:** o push usa as credenciais/configuração de git da sua máquina. Se o remote exigir autenticação interativa e ela não estiver configurada (credential helper/SSH), o push falhará com uma mensagem explicativa em vez de travar.

## 9. Situações especiais e mensagens

### "Nada a replicar" / commit vazio

Significa que o resultado é **idêntico ao branch de destino** — não há diferença para commitar. Causas comuns: o commit já estava aplicado, ou a resolução de conflito manteve o lado **destino**. Não é erro: o branch já reflete o estado desejado e **pode ser enviado (push)** normalmente.

### "A árvore de trabalho possui alterações pendentes"

Ocorre se a cópia tiver mudanças não commitadas antes da replicação. Em uso normal a cópia é limpa; reinicie o processo (clique em **Iniciar** novamente) se necessário.

### TortoiseGit não localizado

Ao precisar resolver um conflito, se o TortoiseGit não for encontrado, o app abre um seletor para você indicar o `TortoiseGitProc.exe` ou `TortoiseGitMerge.exe`. O caminho informado é reutilizado durante a sessão.

### Conflitos ainda pendentes ao concluir

Se algum arquivo ainda contiver marcadores de conflito ( <<<<<<< ), o app não conclui e avisa quais arquivos faltam resolver.



## 10. Perguntas frequentes (FAQ)

### O git.kit altera o meu repositório local?

Não. Ele sempre clona para uma pasta temporária e trabalha nela. Seu repositório original não troca de branch nem tem arquivos modificados.

### Onde fica a cópia de trabalho?

Em `%TEMP%\git.kit\<nome>-<data>-<id>`. O caminho aparece em "Local:" na tela.

### Resolvi o conflito mas diz que o commit ficou vazio. Por quê?

Porque a resolução ficou igual ao destino. Reabra o conflito e combine o lado (**origem**) se a intenção era trazer aquela mudança.

### O push não envia minhas alterações ("Everything up-to-date").

Garanta que concluiu a replicação (botão **Concluir replicação**) antes do push, e que o commit foi de fato criado (veja o Log de comandos).

### Posso replicar para um branch que ainda não existe?

Sim. Digite o nome no campo de destino; ele é criado a partir de develop (sufixo `dev`) ou master.

## 11. Glossário

| Termo                    | Significado                                                      |
|--------------------------|------------------------------------------------------------------|
| <b>Cherry-pick</b>       | Aplicar em um branch um commit específico de outro.              |
| <b>Branch de origem</b>  | Branch que contém o commit a ser replicado.                      |
| <b>Branch de destino</b> | Branch que receberá o commit.                                    |
| <b>Conflito</b>          | Quando o git não consegue mesclar automaticamente as alterações. |
| <b>Stage / git add</b>   | Marcar um arquivo como resolvido/pronto para commit.             |
| <b>Push</b>              | Enviar commits/branches locais para o repositório remoto.        |
| <b>Remote (origin)</b>   | Repositório remoto associado ao projeto.                         |